

that, we're simply going to do the XOR on each 32-bit block of the ciphertext. It would be something like this:

TABLE 4: DECRYPTING THE CIPHERTEXT WITH THE 128-BIT KEY								
Part of ciphertext	FFE1A127	DEC8CC39	4BC88CAC	35DDDBBE	FD6FB26D	DE2BED59	F40490CA	A1609DC1
Part of key	02468ACE	CE02468A	8ACE0246	468ACE02	02468ACE	CE02468A	8ACE0246	468ACE02
Resulting plaintext	FDA72BE9	10CA8AB3	C1068EEA	735715BC	FF2938A3	1029ABD3	7ECA928C	E7EA53C3

We can then concatenate the resulting blocks of plaintext to arrive at the original plaintext. Simple and easy, but was our design really any good?

No. It wasn't. Not only is the encryption easy to break but the design process contains a perspective which is unnecessarily complicated and can be simplified.

How our design process is flawed

While the algorithm does look real good, there is a flaw in its design. The question is where? Let us reconsider the way we're doing the XOR operation. It's in a sequence of blocks. The first 32-bit block of plaintext and the first 32-bit block of the key are being XORed. The second 32-bit block of plaintext and the second 32-bit lock of the key are being XORed. Similar for the 3rd and 4th blocks of plaintext and the key. For the fifth 32-bit block of plaintext, we're reusing the first 32-bit block of the key. If you pay just a little attention, all that we're doing is to XOR the first 128 bits of the plaintext with the full 128-bit key and the next 128 bits of plaintext with the key again. The table we used to show the conversion of plaintext to ciphertext (table 3) could also have been shown as in the following table (table 5).

TABLE 5: SIMPLIFIED VERSION OF ENCRYPTION PROCESS USING 128-BIT BLOCKS OF PLAINTEXT AND COMPLETE 128-BIT KEY (A REFACTOR OF TABLE 3)		
Part of plaintext	FDA72BE910CA8AB3C1068EEA735715BC	FF2938A31029ABD37ECA928CE7EA53C3
Part of key	02468ACECE02468A8ACE0246468ACE02	02468ACECE02468A8ACE0246468ACE02
Resulting ciphertext	FFE1A127DEC8CC394BC88CAC35DDDBBE	FD6FB26DDE2BED59F40490CAA1609DC1

By such behavior we can say that the block size that the algorithm is actually using is not 32 bits but 128 bits. Instead of thinking that we're running the algorithm in chunks of 32 bits, you can rather think that we're